HACKERWOMEN COUNCIL GENERACIÓN 1

SQL & WEB SECURITY CHALLENGE

Actividades realizadas:

SQL Easy — 31 ejercicios de consultas SQL

OWASP ZAP — Análisis de seguridad web

Por: Dalia Montserrat Caballero Silva

FECHA DE ENTREGA:

27 de octubre de 2026

Ejercicio 01. SELECT *

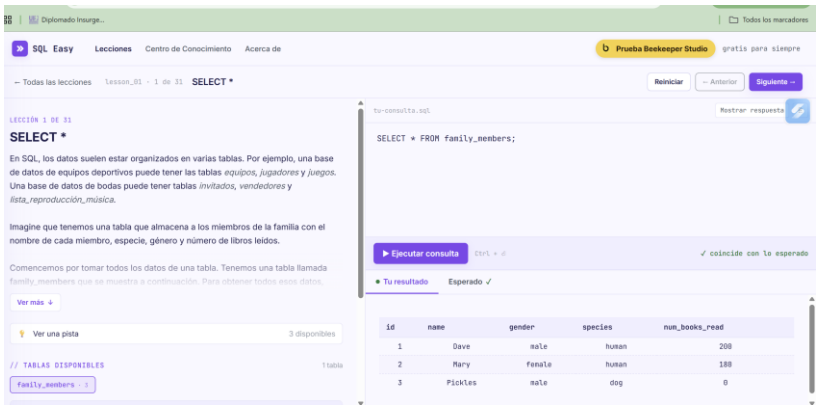
1.Número y nombre: Ejercicio 1 — SELECT *.

2. Objetivo: Obtener todos los registros y columnas de una tabla mediante una consulta SQL.

3. Concepto SQL: La instrucción SELECT * permite seleccionar todas las columnas de una tabla.

4. Consulta ejecutada: SELECT * FROM family_members;

5. Resultado:



6. Interpretación: Muestra la información almacenada en la tabla family_members, incluye identificador, nombre, género, especie y número de libros leídos de cada integrante.

7. Aprendizaje: Usar la sentencia SELECT * para consultar todos los datos de una tabla sin especificar individualmente sus columnas.

8. Aplicación en el mundo real: Puede usarse para consultar información completa de una tabla de usuarios, productos o empleados en un sistema de información

Ejercicio 02. SELECT columnas específicas

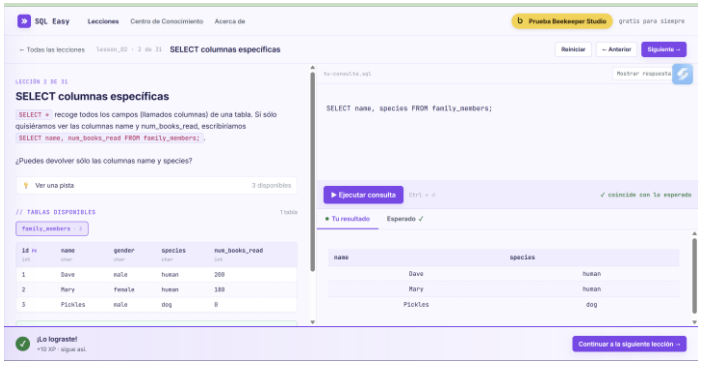
1. Número y nombre: Ejercicio 2 — SELECT columnas específicas.

2. Objetivo: Consultar determinadas columnas de una tabla mediante la instrucción SELECT, evitando mostrar información innecesaria

3. Concepto SQL: La instrucción SELECT permite elegir columnas específicas de una tabla, indicando sus nombres separados por comas, en lugar de utilizar el asterisco (*) para seleccionar todas.

4. Consulta ejecutada: SELECT name, species FROM family_members;

5. Resultado:



6. Interpretación: Muestra solo columnas name y species de la tabla family_members, salen tres registros correspondientes a Dave y Mary, la especie es human, y Pickles es dog.

7. Aprendizaje: Aprendí a seleccionar columnas específicas de una tabla utilizando SELECT, lo que permite obtener solamente la información necesaria y presentar resultados más claros.

8. Aplicación en el mundo real: Puede usarse en sistemas de administración de personal para mostrar únicamente los nombres y puestos de los empleados, sin consultar otras columnas que no sean necesarias.

Ejercicio 03. WHERE ... Equals

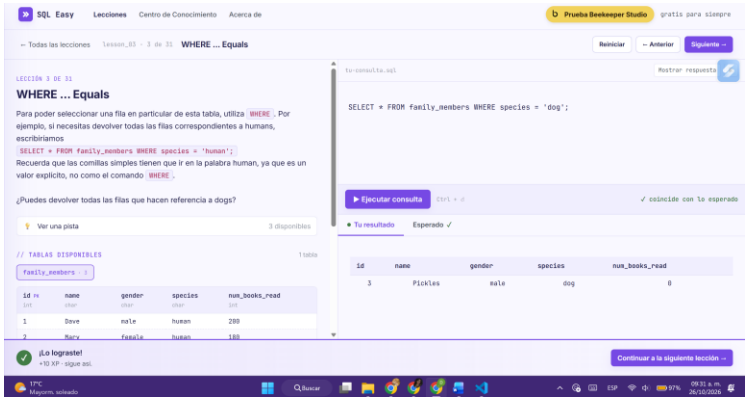
1. Número y nombre: Ejercicio 3 — WHERE ... Equals.

2. Objetivo: Consultar los registros de una tabla que cumplan con una condición específica mediante la cláusula WHERE y el operador de igualdad.

3. Concepto SQL: WHERE permite filtrar registros de una tabla según una condición, operador = compara valores para seleccionar únicamente las filas que coincidan con el valor indicado.

4. Consulta ejecutada: SELECT * FROM family_members WHERE species = 'dog';

5. Resultado:



6. Interpretación: Muestra solo los registros cuya especie es dog, devuelve a Pickles, identificado con el número 3, de género masculino y con cero libros leídos.

7. Aprendizaje: Usar WHERE junto con el operador de igualdad para filtrar información y obtener únicamente los registros que cumplen con una condición determinada.

8. Aplicación en el mundo real: Puede usarse en una tienda para buscar productos de una categoría específica o en sistema escolar para consultar a estudiantes de un grupo determinado.

Ejercicio 04. WHERE ... Mayor que

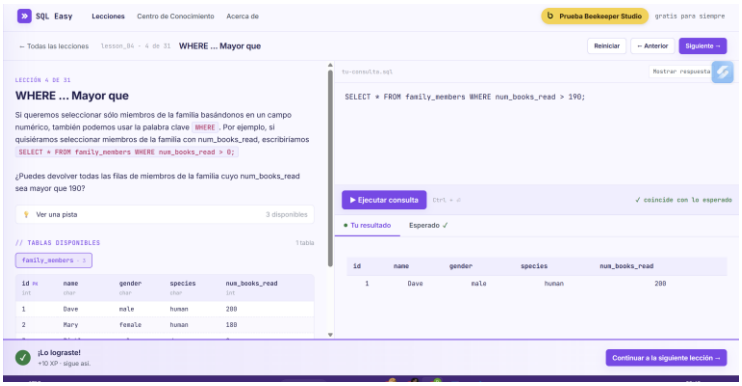
1. Número y nombre: Ejercicio 4 — WHERE ... Mayor que.

2. Objetivo: Consultar registros de una tabla que contengan valores numéricos superiores a una cantidad determinada mediante la cláusula WHERE y el operador de comparación mayor que (>).

3. Concepto SQL: WHERE permite filtrar registros según una condición, operador > se usa para comparar valores numéricos y seleccionar únicamente aquellos que sean mayores que el valor establecido.

4. Consulta ejecutada: SELECT * FROM family_members WHERE num_books_read > 190;

5. Resultado:



6. Interpretación: Regresa el registro de Dave, quien ha leído 200 libros, el resto quedan excluidos porque no cumplen con la condición de haber leído más de 190 libros.

7. Aprendizaje: Usar el operador mayor que (>) dentro de la cláusula WHERE para comparar valores numéricos y obtener registros que superen un límite establecido.

8. Aplicación en el mundo real: Puede usarse en un sistema de ventas para identificar productos cuyo precio sea superior a una cantidad determinada, o en sistema escolar para consultar estudiantes que hayan obtenido calificaciones mayores que un valor establecido.

Ejercicio 05. WHERE ... Mayor o igual que

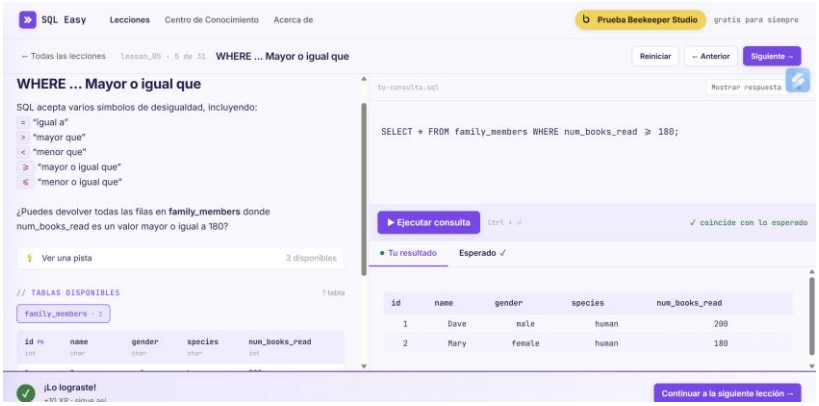
1. Número y nombre: Ejercicio 5 — WHERE ... Mayor o igual que.

2. Objetivo: Consultar registros de una tabla que tengan valores numéricos mayores o iguales a una cantidad determinada, usando WHERE y operador de comparación correspondiente.

3. Concepto SQL: La cláusula WHERE permite filtrar registros según una condición. El operador mayor o igual que (>=) selecciona aquellos registros cuyo valor sea superior o exactamente igual al límite establecido.

4. Consulta ejecutada: SELECT * FROM family_members WHERE num_books_read >= 180;

5. Resultado:



6. Interpretación: Devuelve dos registros de la tabla family_members: Dave que ha leído 200 libros, y Mary que ha leído 180; ambos cumplen con la condición establecida, Pickles queda excluido porque su cantidad de libros leídos es inferior a 180.

7. Aprendizaje: Usar el operador mayor o igual que (>=) dentro de WHERE para filtrar registros numéricos, incluye valores superiores como los que coinciden exactamente con el límite indicado.

8. Aplicación en el mundo real: Puede usarse en un sistema escolar para identificar estudiantes con calificaciones iguales o superiores a la mínima aprobatoria, o en una empresa para consultar productos cuyo inventario sea mayor o igual a una cantidad establecida.

Ejercicio 06. AND

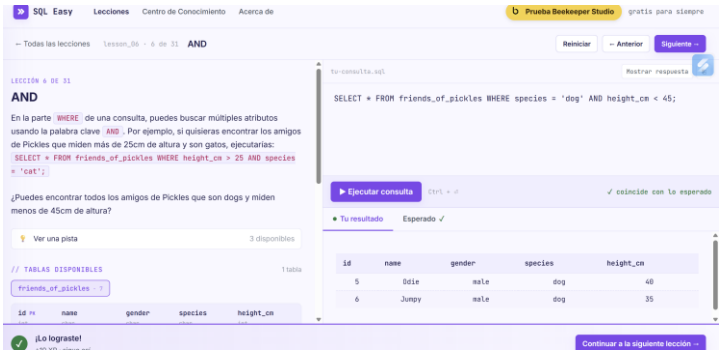
1. Número y nombre: Ejercicio 6 — AND.

2. Objetivo: Consultar los registros de una tabla que cumplan simultáneamente dos condiciones mediante la cláusula WHERE y el operador lógico AND.

3. Concepto SQL: Operador lógico AND permite combinar varias condiciones dentro de una consulta SQL, para que un registro aparezca en el resultado, todas las condiciones establecidas deben cumplirse.

4. Consulta ejecutada: SELECT * FROM friends_of_pickles WHERE species = 'dog' AND height_cm < 45;

5. Resultado:



6. Interpretación: Filtra los registros de la tabla friends_of_pickles, seleccionando los que cuya especie es dog y altura es inferior a 45 centímetros, los que no cumplen ambas condiciones quedan excluidos del resultado.

7. Aprendizaje: Usar el operador lógico AND para combinar dos condiciones dentro de WHERE, permitiendo realizar búsquedas más específicas y obtener los registros que cumplen todos los criterios establecidos.

8. Aplicación en el mundo real: Puede usarse en un sistema de inventario para identificar productos que pertenezcan a una categoría determinada y cuyo precio sea menor a una cantidad establecida, facilitando la búsqueda y selección de información.

Ejercicio 07. OR

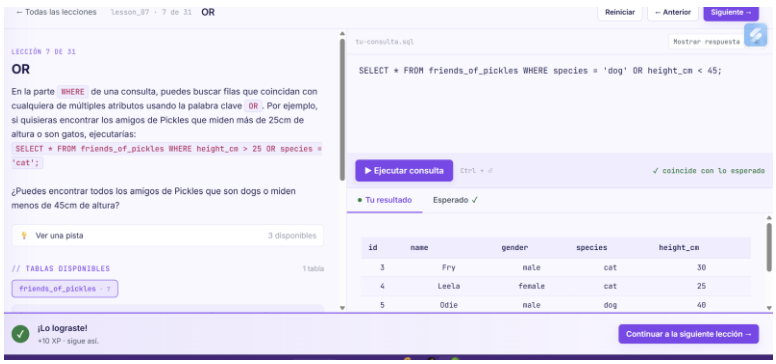
1. Número y nombre: Ejercicio 7 — OR.

2. Objetivo: Consultar registros de una tabla que cumplan al menos una de dos condiciones mediante la cláusula WHERE y el operador lógico OR.

3. Concepto SQL: El operador lógico OR permite combinar varias condiciones dentro de una consulta SQL; un registro se incluye en el resultado cuando cumple al menos una de las condiciones establecidas, sin necesidad de que todas sean verdaderas.

4. Consulta ejecutada: SELECT * FROM friends_of_pickles WHERE species = 'dog' OR height_cm < 45;

5. Resultado:



6. Interpretación: Selecciona los registros de la tabla friends_of_pickles cuya especie sea dog o cuya altura sea inferior a 45 centímetros, permite incluir tanto a los perros de cualquier altura como a los integrantes de otras especies que cumplan la condición de altura.

7. Aprendizaje: Usar el operador lógico OR para combinar condiciones dentro de la cláusula WHERE, permitiendo realizar consultas más flexibles en las que basta con cumplir uno de los criterios establecidos para obtener un registro.

8. Aplicación en el mundo real: Puede usarse en una tienda para buscar productos que pertenezcan a una categoría específica o cuyo precio sea menor que una cantidad determinada, facilitando la búsqueda de productos que cumplan diferentes criterios.

Ejercicio 08. IN

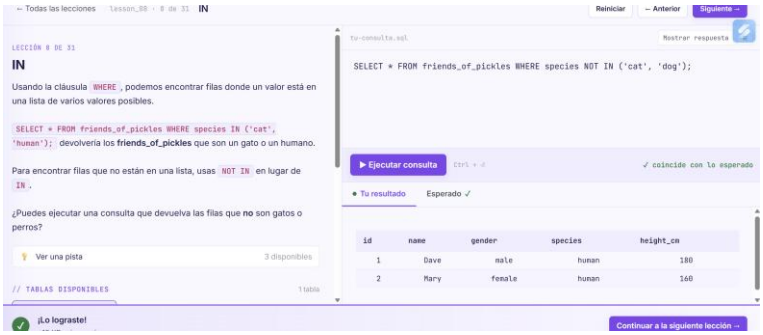
1. Número y nombre: Ejercicio 8 — IN.

2. Objetivo: Consultar los registros de una tabla que no pertenezcan a una lista de valores específicos, utilizando la cláusula WHERE y el operador NOT IN.

3. Concepto SQL: El operador IN permite comprobar si un valor pertenece a una lista de valores definidos, su variante NOT IN permite excluir los registros cuyos valores coincidan con cualquiera de los elementos de la lista.

4. Consulta ejecutada: SELECT * FROM friends_of_pickles WHERE species NOT IN ('cat', 'dog');

5. Resultado:



6. Interpretación: La consulta selecciona los registros de la tabla friends_of_pickles cuya especie no sea cat ni dog, se excluyen los gatos y perros, mostrando únicamente los integrantes que pertenecen a otras especies.

7. Aprendizaje: Usar los operadores IN y NOT IN para comparar valores con una lista de opciones y realizar consultas que permitan incluir o excluir determinados registros de una tabla.

8. Aplicación en el mundo real: Puede usarse en un sistema de inventario para identificar productos que no pertenezcan a ciertas categorías, o en una empresa para consultar empleados que no formen parte de determinados departamentos.

Ejercicio 09. DISTINCT

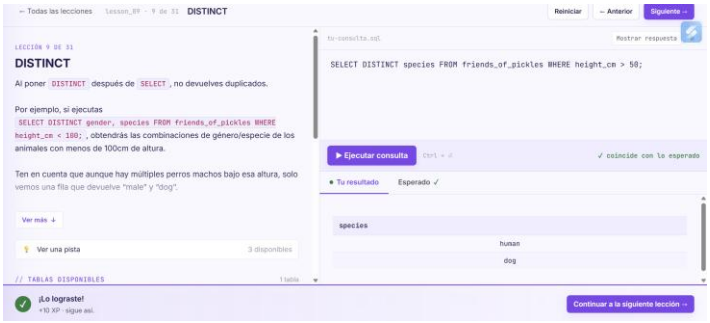
1. Número y nombre: Ejercicio 9 — DISTINCT.

2. Objetivo: Consultar los valores únicos de una columna, eliminando los resultados duplicados y aplicando una condición específica mediante la cláusula WHERE.

3. Concepto SQL: La palabra clave DISTINCT se utiliza junto con SELECT para eliminar los valores duplicados de los resultados de una consulta; obtiene valores diferentes de una o varias columnas, facilitando la identificación de información única dentro de una tabla.

4. Consulta ejecutada: SELECT DISTINCT species FROM friends_of_pickles WHERE height_cm > 50;

5. Resultado:



6. Interpretación: Devuelve dos especies diferentes: human y dog, aunque existen dos humanos que cumplen con la condición de medir más de 50 centímetros, el operador DISTINCT evita repetir la especie human y así se obtiene una lista de especies únicas que cumplen con el filtro establecido.

7. Aprendizaje: Usar DISTINCT para eliminar valores duplicados de los resultados de una consulta SQL y combinarlo con WHERE para obtener información específica sin repeticiones innecesarias.

8. Aplicación en el mundo real: Puede usarse en una tienda para identificar las diferentes categorías de productos disponibles sin repetir sus nombres, o en un sistema escolar para consultar las carreras registradas, mostrando cada una solamente una vez.

Ejercicio 10. ORDER BY

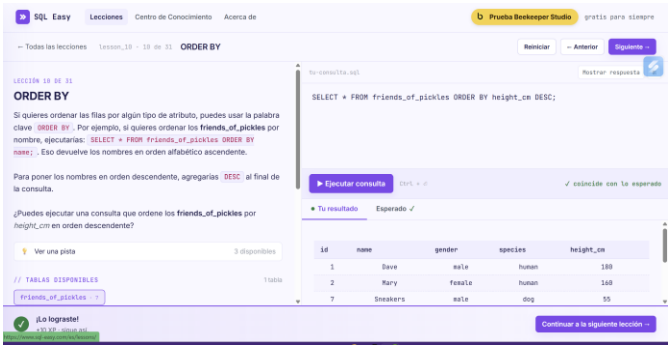
1. Número y nombre: Ejercicio 10 — ORDER BY.

2. Objetivo: Ordenar registros de una tabla según los valores de una columna numérica, utilizando la cláusula ORDER BY en orden descendente.

3. Concepto SQL: La cláusula ORDER BY permite organizar los resultados de una consulta SQL según una o varias columnas, la palabra clave ASC ordena los datos de manera ascendente, mientras que DESC los organiza de manera descendente, es decir, de mayor a menor en valores numéricos.

4. Consulta ejecutada: SELECT * FROM friends_of_pickles ORDER BY height_cm DESC;

5. Resultado:



6. Interpretación: La consulta devuelve todos los registros de la tabla friends_of_pickles, organizados según la columna height_cm en orden descendente, los integrantes con mayor altura aparecen primero, mientras que los de menor altura se muestran al final, facilitando la comparación de sus estaturas.

7. Aprendizaje: Usar la cláusula ORDER BY para ordenar los resultados de una consulta SQL y a emplear DESC para organizar valores numéricos de mayor a menor, permitiendo presentar la información de forma estructurada.

8. Aplicación en el mundo real: Puede usarse en un sistema de ventas para mostrar los productos ordenados del más caro al más económico, o en un sistema escolar para organizar las calificaciones de los estudiantes de mayor a menor.

Ejercicio 11. LIMIT — Número de filas devueltas

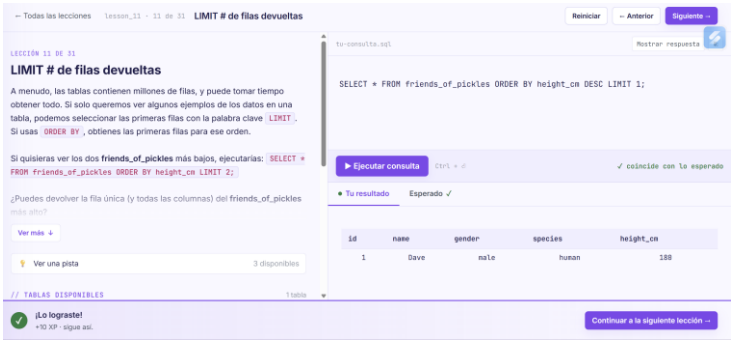
1. Número y nombre: Ejercicio 11 — LIMIT, número de filas devueltas.

2. Objetivo: Consultar únicamente el registro con el valor más alto de una columna numérica, utilizando las cláusulas ORDER BY y LIMIT para ordenar y limitar los resultados.

3. Concepto SQL: La cláusula LIMIT permite restringir la cantidad de filas devueltas por una consulta SQL, al combinarla con ORDER BY DESC, es posible ordenar los registros de mayor a menor y seleccionar solamente los primeros resultados de acuerdo con el límite establecido.

4. Consulta ejecutada: SELECT * FROM friends_of_pickles ORDER BY height_cm DESC LIMIT 1;

5. Resultado:



6. Interpretación: La consulta ordena los registros de la tabla friends_of_pickles según su altura en centímetros, de mayor a menor y mediante LIMIT 1, devuelve únicamente el registro del integrante más alto, mostrando todas las columnas correspondientes a su información.

7. Aprendizaje: Usar la cláusula LIMIT para controlar la cantidad de registros mostrados en una consulta SQL y a combinarla con ORDER BY DESC para identificar y recuperar el registro con el valor más alto de una columna.

8. Aplicación en el mundo real: Puede usarse en un sistema de ventas para identificar el producto más caro de una tienda, o en un sistema escolar para consultar al estudiante que obtuvo la calificación más alta.

Ejercicio 12. COUNT(*)

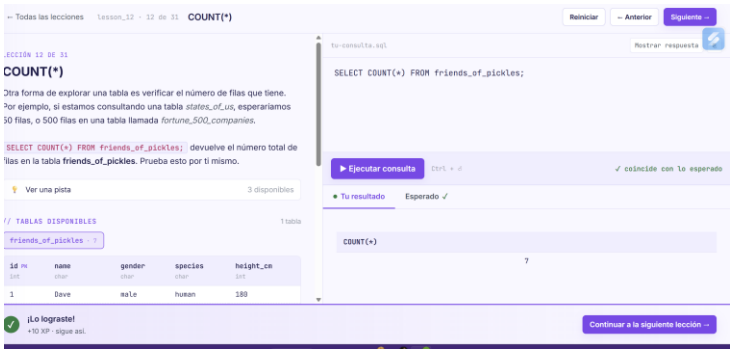
1. Número y nombre: Ejercicio 12 — COUNT(*).

2. Objetivo: Determinar la cantidad total de registros almacenados en una tabla mediante la función de agregación COUNT(*).

3. Concepto SQL: La función COUNT(*) permite contar el número total de filas de una tabla, incluyendo aquellas que puedan contener valores nulos en alguna de sus columnas, es una función de agregación utilizada para obtener información estadística sobre los registros.

4. Consulta ejecutada: SELECT COUNT(*) FROM friends_of_pickles;

5. Resultado:



6. Interpretación: La consulta devuelve como resultado el número 7, lo que indica que la tabla friends_of_pickles contiene un total de siete registros, la función COUNT(*) proporciona únicamente la cantidad total de filas almacenadas.

7. Aprendizaje: Usar la función COUNT(*) para contar los registros de una tabla SQL sin necesidad de consultar cada fila individualmente, facilitando la obtención de información cuantitativa sobre una base de datos.

8. Aplicación en el mundo real: Puede usarse en un sistema escolar para conocer el número total de estudiantes registrados, en una tienda para contar los productos almacenados o en una empresa para determinar la cantidad total de empleados.

Ejercicio 13. COUNT(*) ... WHERE

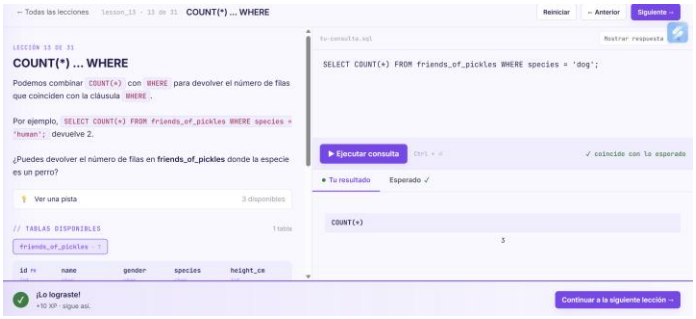
1. Número y nombre: Ejercicio 13 — COUNT(*) ... WHERE.

2. Objetivo: Contar los registros de una tabla que cumplan con una condición específica, utilizando la función COUNT(*) junto con la cláusula WHERE.

3. Concepto SQL: La función COUNT(*) permite contar el número de filas de una tabla, mientras que la cláusula WHERE permite establecer condiciones para filtrar los registros, combinar ambas instrucciones, es posible obtener la cantidad de registros que cumplen con un criterio determinado.

4. Consulta ejecutada: SELECT COUNT(*) FROM friends_of_pickles WHERE species = 'dog';

5. Resultado:



6. Interpretación: La consulta cuenta únicamente los registros de la tabla friends_of_pickles cuya especie es dog, los registros correspondientes a humanos, gatos u otras especies no se incluyen en el conteo, el resultado representa la cantidad total de perros registrados en la tabla.

7. Aprendizaje: Aprendí a combinar la función COUNT(*) con la cláusula WHERE para contar registros que cumplen con una condición específica, lo que permite realizar consultas estadísticas más precisas dentro de una base de datos.

8. Aplicación en el mundo real: Puede usarse en un sistema escolar para conocer cuántos estudiantes pertenecen a una carrera determinada, o en una tienda para contar los productos disponibles de una categoría específica.

Ejercicio 14. SUM

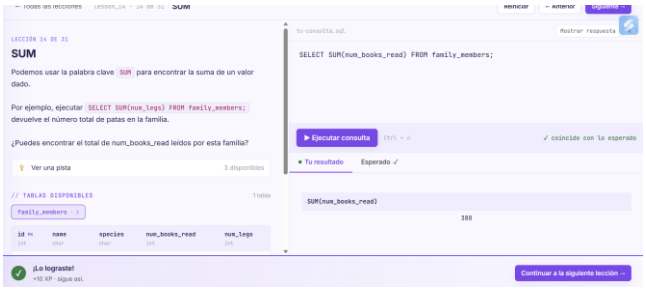
1. Número y nombre: Ejercicio 14 — SUM.

2. Objetivo: Calcular la suma total de los valores numéricos almacenados en una columna de una tabla, utilizando la función de agregación SUM().

3. Concepto SQL: La función SUM() permite sumar los valores numéricos de una columna y devolver un resultado total, es una función de agregación que facilita la realización de operaciones matemáticas sobre los registros de una base de datos.

4. Consulta ejecutada: SELECT SUM(num_books_read) FROM family_members;

5. Resultado:



6. Interpretación: La consulta devuelve como resultado el valor 380, que representa la cantidad total de libros leídos por los integrantes de la tabla family_members; Dave ha leído 200 libros, Mary 180 y Pickles 0 y con la función SUM(), se obtiene el total de 380 libros.

7. Aprendizaje: Usar la función SUM() para realizar operaciones de suma sobre los valores numéricos de una columna, permitiendo obtener totales sin necesidad de calcular manualmente cada registro.

8. Aplicación en el mundo real: Puede usarse en un sistema de ventas para calcular el total de ingresos generados por una empresa, en un sistema de inventario para sumar las existencias de productos o en una biblioteca para conocer la cantidad total de libros prestados.

Ejercicio 15. AVG

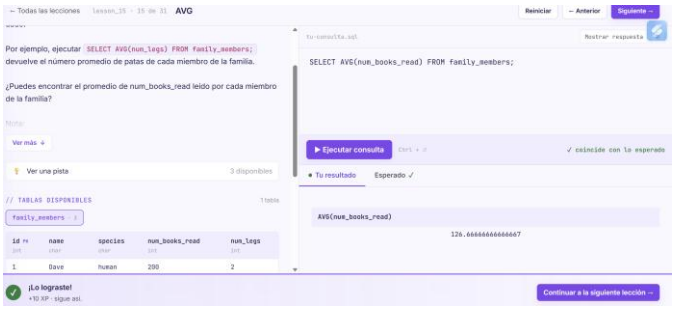
1. Número y nombre: Ejercicio 15 — AVG.

2. Objetivo: Calcular el promedio de los valores numéricos almacenados en una columna de una tabla, utilizando la función de agregación AVG().

3. Concepto SQL: La función AVG() permite calcular la media aritmética de los valores numéricos de una columna, es una función de agregación que suma los valores disponibles y los divide entre la cantidad de registros con valores no nulos, facilitando la obtención de promedios dentro de una base de datos.

4. Consulta ejecutada: SELECT AVG(num_books_read) FROM family_members;

5. Resultado:



6. Interpretación: Devuelve un promedio aproximado de 126.67 libros leídos por integrante de la tabla family_members, Dave ha leído 200 libros, Mary 180 y Pickles 0, AVG() suma estos valores y divide el resultado entre los tres integrantes, obteniendo el promedio de libros leídos por cada miembro.

7. Aprendizaje: Usar la función AVG() para calcular el promedio de los valores numéricos de una columna, permitiendo obtener información estadística sin necesidad de realizar operaciones matemáticas manualmente.

8. Aplicación en el mundo real: Puede usarse en un sistema escolar para calcular el promedio de calificaciones de los estudiantes, en una empresa para conocer el salario promedio de sus empleados o en un sistema de ventas para determinar el promedio de ingresos por transacción.

Ejercicio 16. MAX y MIN

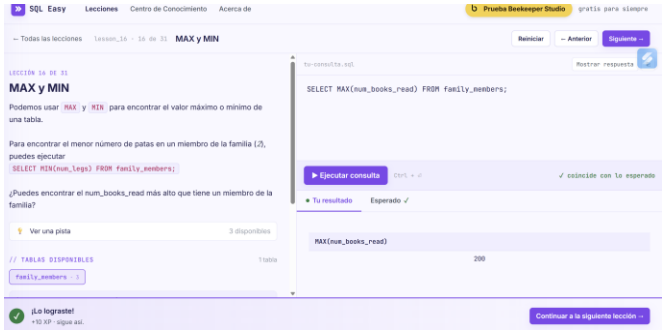
1. Número y nombre: Ejercicio 16 — MAX y MIN.

2. Objetivo: Identificar el valor numérico más alto de una columna de una tabla mediante la función de agregación MAX().

3. Concepto SQL: Las funciones MAX() y MIN() permiten identificar los valores máximo y mínimo de una columna, respectivamente; MAX() devuelve el valor más alto encontrado, mientras que MIN() devuelve el valor más bajo, ambas son funciones de agregación utilizadas para analizar información almacenada en una base de datos.

4. Consulta ejecutada: SELECT MAX(num_books_read) FROM family_members;

5. Resultado:



6. Interpretación: Devuelve el valor 200, que representa la mayor cantidad de libros leídos por un integrante de la tabla family_members, al comparar los valores registrados, Dave ha leído 200 libros, Mary 180 y Pickles 0, por lo que el valor máximo corresponde a Dave.

7. Aprendizaje: Usar funciones MAX() y MIN() para identificar valores extremos de una columna, apliqué MAX() para obtener valor numérico más alto sin ordenar o revisar manualmente todos los registros.

8. Aplicación en el mundo real: Puede usarse en un sistema escolar para identificar la calificación más alta obtenida por los estudiantes, en una empresa para determinar el salario máximo registrado o en un sistema de ventas para consultar el precio más elevado de los productos.

Ejercicio 17. GROUP BY

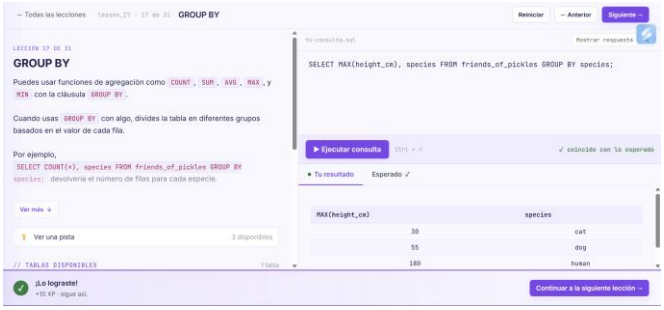
1. Número y nombre: Ejercicio 17 — GROUP BY.

2. Objetivo: Agrupar los registros de una tabla según una columna específica y obtener el valor máximo de cada grupo mediante la combinación de GROUP BY y MAX().

3. Concepto SQL: GROUP BY permite organizar los registros de una tabla en grupos que comparten un mismo valor, puede combinarse con funciones de agregación como COUNT(), SUM(), AVG(), MAX() y MIN() para realizar cálculos independientes sobre cada grupo.

4. Consulta ejecutada: SELECT MAX(height_cm), species FROM friends_of_pickles GROUP BY species;

5. Resultado:



6. Interpretación: Agrupa los registros de la tabla friends_of_pickles según su especie y obtiene la altura máxima de cada grupo, se muestra una sola fila por especie, acompañada del valor más alto registrado en la columna height_cm, permitiendo comparar las mayores alturas entre las diferentes especies.

7. Aprendizaje: Usar la cláusula GROUP BY para agrupar registros que comparten características comunes y combinarla con la función MAX() para obtener el valor máximo de cada grupo, facilitando el análisis y la organización de información.

8. Aplicación en el mundo real: Puede usarse en un sistema escolar para identificar la calificación más alta de cada grupo de estudiantes, en una empresa para conocer el salario máximo de cada departamento o en un sistema de ventas para determinar el precio más elevado de los productos por categoría.

Ejercicio 18. Consultas anidadas

1. Número y nombre: Ejercicio 18 — Consultas anidadas.

2. Objetivo: Consultar los registros de una tabla que contengan el valor máximo de una columna numérica, utilizando una subconsulta dentro de la cláusula WHERE.

3. Concepto SQL: También conocidas como subconsultas, permiten ejecutar una consulta SQL dentro de otra; la consulta interna obtiene un resultado que posteriormente es utilizado por la consulta externa para filtrar o seleccionar información específica de una base de datos.

4. Consulta ejecutada: SELECT * FROM family_members WHERE num_books_read = (SELECT MAX(num_books_read) FROM family_members);

5. Resultado:



6. Interpretación: Devuelve el registro de Dave, quien ha leído 200 libros, es la cantidad más alta registrada en la tabla family_members, la subconsulta utiliza la función MAX() para identificar el valor máximo de la columna num_books_read, la consulta principal selecciona los registros que coinciden con dicho valor, mostrando toda la información del integrante correspondiente.

7. Aprendizaje: Usar consultas anidadas para combinar dos operaciones SQL dentro de una misma instrucción, comprendí cómo una subconsulta puede obtener un valor que posteriormente se utiliza como condición para seleccionar registros específicos, permitiendo realizar búsquedas más complejas.

8. Aplicación en el mundo real: Puede usarse en un sistema escolar para identificar a los estudiantes que obtuvieron la calificación más alta, en una empresa para consultar a los empleados con el salario máximo o en un sistema de ventas para encontrar los productos con el precio más elevado.

Ejercicio 19. NULL

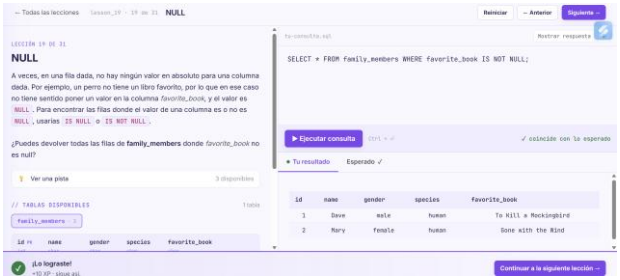
1. Número y nombre: Ejercicio 19 — NULL.

2. Objetivo: Consultar los registros de una tabla que contengan valores no nulos en una columna específica, utilizando la cláusula WHERE y el operador IS NOT NULL.

3. Concepto SQL: NULL representa la ausencia de un valor en una columna, el operador IS NULL permite identificar registros que no tienen un valor registrado, IS NOT NULL selecciona aquellos que sí contienen información, estos operadores permiten gestionar y consultar datos incompletos dentro de una BD.

4. Consulta ejecutada: SELECT * FROM family_members WHERE favorite_book IS NOT NULL;

5. Resultado:



6. Interpretación: La consulta devuelve los registros de Dave y Mary, quienes tienen información registrada en la columna favorite_book, Dave tiene como libro favorito To Kill a Mockingbird y Mary Gone with the Wind, Pickles queda excluido porque no tiene un valor registrado en dicha columna.

7. Aprendizaje: Usar IS NULL e IS NOT NULL para identificar registros con información ausente o disponible en una columna, comprendí que NULL representa la ausencia de un valor y que debe usarse una condición específica para realizar consultas relacionadas con estos datos.

8. Aplicación en el mundo real: Puede usarse en un sistema escolar para identificar estudiantes que tienen registrado su correo electrónico, en una empresa para consultar empleados que cuentan con un número telefónico o en una tienda en línea para localizar productos que tienen una descripción registrada.

Ejercicio 20. Fecha

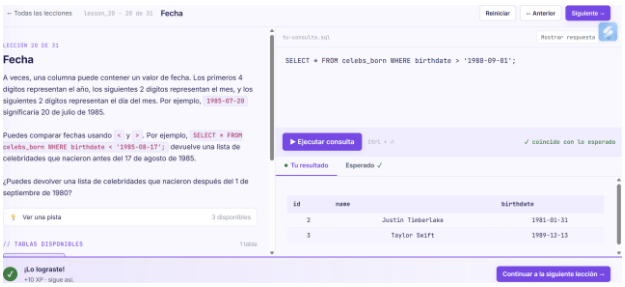
1. Número y nombre: Ejercicio 20 — Fecha.

2. Objetivo: Consultar los registros de una tabla que contengan fechas posteriores a una fecha determinada, utilizando la cláusula WHERE y el operador de comparación mayor que.

3. Concepto SQL: Fechas pueden representarse mediante el formato AAAA-MM-DD, que corresponde al año, mes y día; cuando las fechas se encuentran almacenadas correctamente en este formato, es posible utilizar operadores de comparación como >, <, >= y <= para filtrar registros según un periodo determinado.

4. Consulta ejecutada: SELECT * FROM celebs_born WHERE birthdate > '1980-09-01';

5. Resultado:



6. Interpretación: Selecciona registros de la tabla celebs_born con fecha de nacimiento posterior al 1 de septiembre de 1980 y por la condición birthdate > '1980-09-01', se excluyen las celebridades nacidas antes de esa fecha o exactamente ese día, mostrando solo los que cumplen con el criterio establecido.

7. Aprendizaje: Aprendí a realizar consultas SQL utilizando fechas como condiciones de búsqueda, comprendí cómo emplear el formato de fecha año-mes-día y los operadores de comparación para identificar registros correspondientes a periodos específicos.

8. Aplicación en el mundo real: Puede usarse en un sistema escolar para identificar estudiantes inscritos después de una fecha determinada, en una empresa para consultar empleados contratados a partir de cierto periodo o en un sistema de ventas para localizar transacciones realizadas después de una fecha específica.

Ejercicio 21. Inner joins

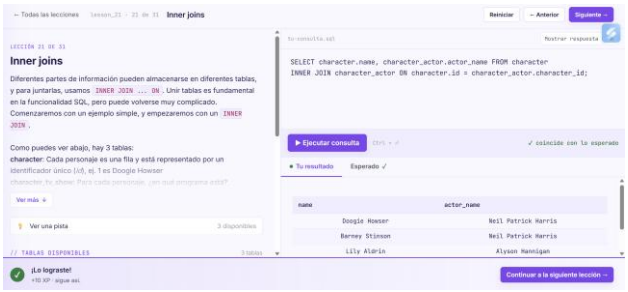
1. Número y nombre: Ejercicio 21 — Inner joins.

2. Objetivo: Relacionar registros de dos tablas mediante una columna en común, utilizando la cláusula INNER JOIN para obtener información combinada de los personajes y los actores que los interpretan.

3. Concepto SQL: INNER JOIN permite combinar registros de dos o más tablas que comparten una relación mediante columnas comunes, la condición ON establece qué columnas deben coincidir para realizar la unión, este JOIN devuelve solo los registros que tienen correspondencia en ambas tablas.

4. Consulta ejecutada: SELECT character.name, character_actor.actor_name FROM character INNER JOIN character_actor ON character.id = character_actor.character_id;

5. Resultado:



6. Interpretación: Relaciona las tablas character y character_actor mediante las columnas id y character_id y se obtienen cuatro registros que muestran los personajes y sus actores; Doogie Howser y Barney Stinson son interpretados por Neil Patrick Harris, y Lily Aldrin y Willow Rosenberg son interpretadas por Alyson Hannigan, la unión permite consultar información relacionada que originalmente se encuentra almacenada en tablas diferentes.

7. Aprendizaje: INNER JOIN para combinar información de dos tablas relacionadas mediante una columna en común; comprendí la importancia de establecer correctamente la condición ON para obtener registros que coincidan entre ambas tablas, facilitando las consultas en BD relacionales.

8. Aplicación en el mundo real: Puede usarse en un sistema escolar para relacionar estudiantes con sus respectivas carreras, en una empresa para asociar empleados con sus departamentos o en un sistema de ventas para consultar productos junto con la información de sus proveedores.

Ejercicio 22. Múltiples joins

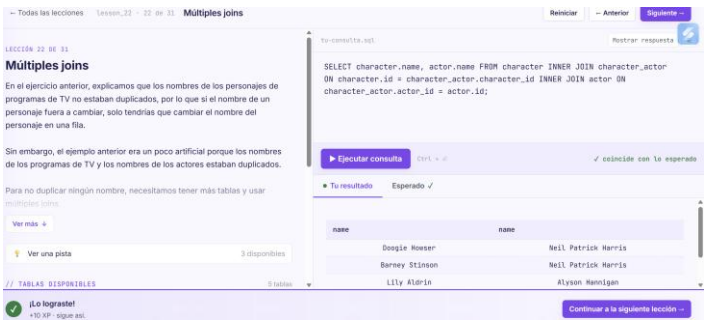
1. Número y nombre: Ejercicio 22 — Múltiples joins.

2. Objetivo: Relacionar información de tres tablas mediante múltiples operaciones INNER JOIN, utilizando columnas en común para obtener los nombres de los personajes y los actores que los interpretan.

3. Concepto SQL: La utilización de múltiples JOIN permite combinar información almacenada en tres o más tablas relacionadas, por INNER JOIN y las condiciones ON, se establecen las relaciones entre las tablas para obtener registros que coincidan, evitando almacenar información duplicada y facilitando las consultas en bases de datos relacionales.

4. Consulta ejecutada: SELECT character.name, actor.name FROM character INNER JOIN character_actor ON character.id = character_actor.character_id INNER JOIN actor ON character_actor.actor_id = actor.id;

5. Resultado:



6. Interpretación: Relaciona tablas character, character_actor y actor por dos operaciones INNER JOIN, y obtienen cuatro registros que muestran nombres de los personajes junto con sus actores, Doogie Howser y Barney Stinson son interpretados por Neil Patrick Harris, mientras que Lily Aldrin y Willow Rosenberg son interpretadas por Alyson Hannigan, esto permite consultar información relacionada almacenada en diferentes tablas.

7. Aprendizaje: Usar múltiples operaciones INNER JOIN para relacionar información de tres tablas dentro de una misma consulta SQL, comprendí cómo las tablas intermedias permiten establecer relaciones mediante identificadores y cómo utilizar las condiciones ON para combinar correctamente los registros.

8. Aplicación en el mundo real: Puede usarse en un sistema escolar para relacionar estudiantes, inscripciones y materias; en una empresa para consultar empleados, departamentos y proyectos; o en un sistema de ventas para relacionar pedidos, productos y clientes.

Ejercicio 23. Joins con WHERE

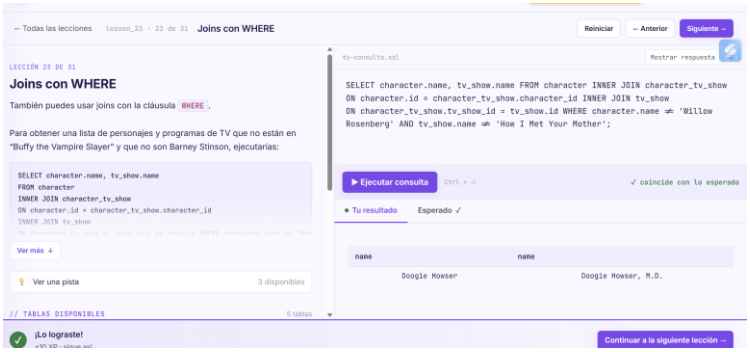
1. Número y nombre: Ejercicio 23 — Joins con WHERE.

2. Objetivo: Relacionar información de diferentes tablas por operaciones INNER JOIN y aplicar condiciones de filtrado utilizando WHERE, para obtener los registros que cumplan con los criterios establecidos.

3. Concepto SQL: INNER JOIN permite combinar registros de diferentes tablas por columnas relacionadas, y WHERE permite filtrar resultados según condiciones específicas, el operador != se utiliza para excluir valores determinados y AND permite combinar varias condiciones que deben cumplirse simultáneamente.

4. Consulta ejecutada: SELECT character.name, tv_show.name FROM character INNER JOIN character_tv_show ON character.id = character_tv_show.character_id INNER JOIN tv_show ON character_tv_show.tv_show_id = tv_show.id WHERE character.name != 'Willow Rosenberg' AND tv_show.name != 'How I Met Your Mother';

5. Resultado:



6. Interpretación: Relaciona tablas character, character_tv_show y tv_show por dos operaciones INNER JOIN, después usa WHERE para excluir al personaje Willow Rosenberg y los registros pertenecientes al programa How I Met Your Mother, y se obtiene únicamente el personaje Doogie Howser, perteneciente al programa Doogie Howser, M.D., ya que cumple con ambas condiciones establecidas.

7. Aprendizaje: Combinar operaciones INNER JOIN con WHERE para consultar información relacionada entre varias tablas y aplicar filtros específicos, comprendí cómo utilizar los operadores de desigualdad y el operador lógico AND para excluir registros que no cumplen con las condiciones establecidas.

8. Aplicación en el mundo real: Puede usarse en un sistema escolar para consultar estudiantes y materias excluyendo determinados grupos o asignaturas, en una empresa para relacionar empleados y departamentos aplicando restricciones específicas, o en un sistema de ventas para consultar productos y proveedores excluyendo determinadas categorías.

Ejercicio 24. Left joins

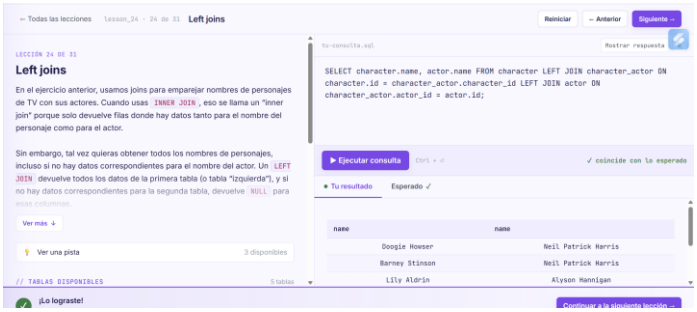
1. Número y nombre: Ejercicio 24 — Left joins.

2. Objetivo: Relacionar información de diferentes tablas mediante operaciones LEFT JOIN, permitiendo obtener todos los registros de la tabla principal, incluso cuando no existan datos relacionados en las demás tablas.

3. Concepto SQL: LEFT JOIN permite combinar información de dos o más tablas relacionadas, conservando todos los registros de la tabla izquierda, cuando no existe una coincidencia en la tabla derecha, los campos correspondientes se muestran con el valor NULL, esto permite consultar información relacionada sin excluir los registros que carecen de una correspondencia.

4. Consulta ejecutada: SELECT character.name, actor.name FROM character LEFT JOIN character_actor ON character.id = character_actor.character_id LEFT JOIN actor ON character_actor.actor_id = actor.id;

5. Resultado:



6. Interpretación: Relaciona las tablas character, character_actor y actor por dos operaciones LEFT JOIN, y se obtienen seis registros correspondientes a los personajes disponibles, Doogie Howser y Barney Stinson están relacionados con Neil Patrick Harris, y Lily Aldrin y Willow Rosenberg están relacionados con Alyson Hannigan; los personajes Steve Urkel y Homer Simpson también aparecen en el resultado, pero con el valor NULL en la columna del actor, debido a que no tienen información relacionada en las tablas utilizadas.

7. Aprendizaje: LEFT JOIN para relacionar información de diferentes tablas sin perder los registros de la tabla principal; comprendí que a diferencia de INNER JOIN, esta operación permite incluir registros que no tienen coincidencias en las demás tablas, mostrando valores NULL cuando no existe información relacionada.

8. Aplicación en el mundo real: Puede usarse en un sistema escolar para mostrar todos los estudiantes, incluso aquellos que todavía no tienen materias asignadas; en una empresa para consultar empleados con o sin departamento registrado; o en un sistema de ventas para identificar clientes que aún no han realizado compras.

Ejercicio 25. Alias de tabla

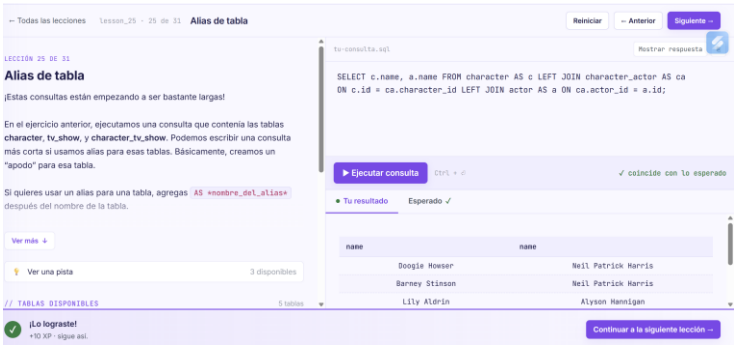
1. Número y nombre: Ejercicio 25 — Alias de tabla.

2. Objetivo: Simplificar la escritura de consultas SQL que relacionan varias tablas mediante la utilización de alias, combinando operaciones LEFT JOIN para obtener los nombres de los personajes y los actores que los interpretan.

3. Concepto SQL: Los alias de tabla permiten asignar nombres temporales y abreviados a las tablas mediante la palabra clave AS, esos nombres se utilizan dentro de una consulta para identificar las columnas y simplificar las instrucciones SQL, los alias son especialmente útiles cuando se realizan varias operaciones JOIN, ya que permiten escribir consultas más cortas, organizadas y fáciles de comprender.

4. Consulta ejecutada: SELECT c.name, a.name FROM character AS c LEFT JOIN character_actor AS ca ON c.id = ca.character_id LEFT JOIN actor AS a ON ca.actor_id = a.id;

5. Resultado:



6. Interpretación: La consulta relaciona las tablas character, character_actor y actor mediante dos operaciones LEFT JOIN, usando los alias c, ca y a para simplificar sus nombres y así se obtienen seis registros que muestran los personajes y los actores que los interpretan, Doogie Howser y Barney Stinson están relacionados con Neil Patrick Harris, Lily Aldrin y Willow Rosenberg están relacionadas con Alyson Hannigan, Steve Urkel y Homer Simpson aparecen con el valor NULL debido a que no tienen un actor relacionado en las tablas utilizadas.

7. Aprendizaje: Usar la palabra clave AS para asignar alias temporales a las tablas dentro de una consulta SQL; comprendí que estos alias permiten reducir la longitud de las instrucciones y facilitan la identificación de columnas cuando se realizan múltiples operaciones JOIN, sin modificar los nombres originales de las tablas.

8. Aplicación en el mundo real: Puede usarse en un sistema escolar para relacionar estudiantes, materias e inscripciones mediante alias que simplifiquen las consultas; en una empresa para consultar empleados y departamentos; o en un sistema de ventas para relacionar clientes, pedidos y productos de manera más organizada.

Ejercicio 26. Alias de columna

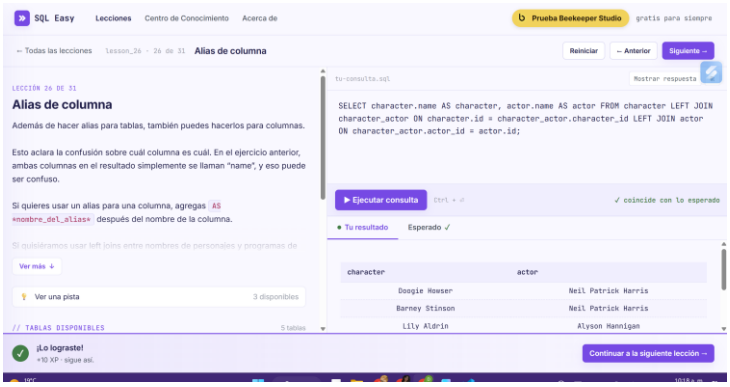
1. Número y nombre: Ejercicio 26 — Alias de columna.

2. Objetivo: Asignar nombres temporales a las columnas de una consulta SQL mediante la palabra clave AS, utilizando operaciones LEFT JOIN para relacionar los nombres de los personajes con los actores que los interpretan.

3. Concepto SQL: Los alias de columna permiten asignar nombres temporales a las columnas de una consulta mediante la palabra clave AS, esos nombres se muestran en resultados sin modificar la estructura original de la base de datos, su uso facilita la identificación de información cuando varias tablas contienen columnas con nombres iguales, mejorando la claridad y comprensión de los resultados.

4. Consulta ejecutada: SELECT character.name AS character, actor.name AS actor FROM character LEFT JOIN character_actor ON character.id = character_actor.character_id LEFT JOIN actor ON character_actor.actor_id = actor.id;

5. Resultado:



6. Interpretación: Relaciona las tablas character, character_actor y actor mediante dos operaciones LEFT JOIN, y se obtienen seis registros que muestran los nombres de los personajes y sus respectivos actores, por palabra clave AS, las columnas se identifican como character y actor, evitando confusiones entre los campos que originalmente comparten el nombre name, los personajes que no tienen un actor relacionado aparecen con el valor NULL.

7. Aprendizaje: Usar palabra clave AS para asignar alias temporales a las columnas de una consulta SQL, comprendí que esta herramienta permite presentar resultados más claros y organizados, especialmente cuando se combinan varias tablas que contienen columnas con nombres similares o idénticos.

8. Aplicación en el mundo real: Puede usarse en un sistema escolar para mostrar columnas identificadas como estudiante y carrera, en una empresa para diferenciar los nombres de empleados y departamentos, o en un sistema de ventas para presentar los nombres de productos y proveedores de manera más clara.

Ejercicio 27. Self joins

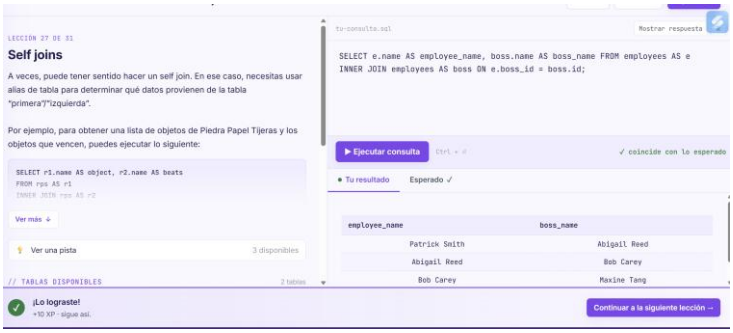
1. Número y nombre: Ejercicio 27 — Self joins.

2. Objetivo: Relacionar registros de una misma tabla mediante una operación SELF JOIN, utilizando alias para identificar a los empleados y sus respectivos jefes dentro de una organización.

3. Concepto SQL: SELF JOIN permite relacionar una tabla consigo misma mediante una cláusula JOIN, para diferenciar los registros, se usan alias que representan distintas referencias a la misma tabla, esta técnica permite consultar relaciones jerárquicas, como empleados y supervisores, sin necesidad de almacenar la información en tablas separadas.

4. Consulta ejecutada: SELECT e.name AS employee_name, boss.name AS boss_name FROM employees AS e INNER JOIN employees AS boss ON e.boss_id = boss.id;

5. Resultado:



6. Interpretación: La consulta relaciona la tabla employees consigo misma mediante una operación INNER JOIN, y se obtienen tres registros que muestran a cada empleado junto con su jefe correspondiente, Patrick Smith tiene como jefa a Abigail Reed, Abigail Reed tiene como jefe a Bob Carey y Bob Carey tiene como jefa a Maxine Tang, y por los alias e y boss, es posible diferenciar los registros de empleados y jefes, aunque ambos pertenecen a la misma tabla.

7. Aprendizaje: SELF JOIN para relacionar registros dentro de una misma tabla SQL, comprendí la importancia de utilizar alias para diferenciar los registros y establecer relaciones jerárquicas mediante identificadores, permitiendo obtener información relacionada sin utilizar múltiples tablas.

8. Aplicación en el mundo real: Puede usarse en una empresa para identificar a los empleados y sus supervisores, en un sistema escolar para relacionar profesores con sus coordinadores académicos o en una organización para consultar la estructura jerárquica de sus departamentos.

Ejercicio 28. LIKE

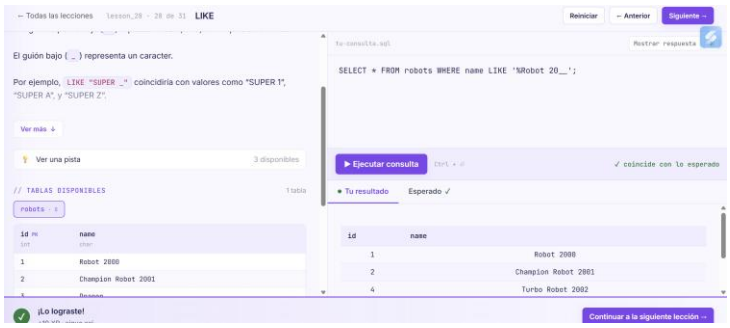
1. Número y nombre: Ejercicio 28 — LIKE.

2. Objetivo: Consultar los registros de una tabla que coincidan con un patrón de texto específico, utilizando el operador LIKE y los caracteres comodín para identificar nombres de robots asociados con años entre 2000 y 2099.

3. Concepto SQL: LIKE permite realizar búsquedas de texto mediante patrones dentro de una consulta SQL, se usa junto con la cláusula WHERE y admite caracteres especiales como el porcentaje (%), que representa cero o más caracteres, y el guion bajo (_), que representa exactamente un carácter, eso permite identificar registros que coincidan con determinadas estructuras de texto.

4. Consulta ejecutada: SELECT * FROM robots WHERE name LIKE '%Robot 20__';

5. Resultado:



6. Interpretación: La consulta devuelve cinco registros de la tabla robots cuyos nombres coinciden con el patrón establecido, se obtienen Robot 2000, Champion Robot 2001, Turbo Robot 2002, Super Robot 2003 y Super Turbo Robot 2004, y % permite que existan palabras antes de Robot, mientras que 20__ identifica un segmento que comienza con 20 y termina con dos caracteres adicionales, de esta manera, se excluyen los nombres que no cumplen con el patrón solicitado.

7. Aprendizaje: LIKE se usa para realizar búsquedas de texto mediante patrones específicos, comprendí cómo funcionan los caracteres comodín % y _, así como su utilidad para localizar registros sin necesidad de conocer el contenido completo de una columna.

8. Aplicación en el mundo real: Puede usarse en un sistema de inventario para localizar productos cuyos nombres contengan determinadas palabras, en una empresa para buscar códigos de artículos que sigan un formato específico o en un sistema de gestión documental para identificar archivos mediante patrones de nombres.

Ejercicio 29. CASE

1. Número y nombre: Ejercicio 29 — CASE.

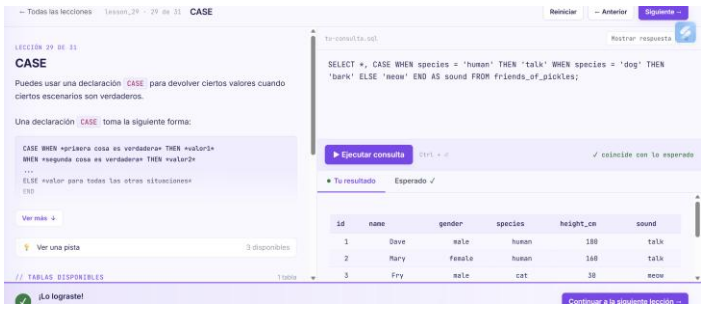
2. Objetivo: Aplicar condiciones dentro de una consulta SQL mediante la expresión CASE para asignar diferentes valores a una nueva columna, dependiendo de la información registrada en cada fila de una tabla.

3. Concepto SQL: La expresión CASE permite evaluar diferentes condiciones y devolver un valor específico cuando alguna de ellas se cumple, usa las palabras clave WHEN para establecer condiciones, THEN para indicar el valor que devolverá, ELSE para definir un resultado alternativo y END para finalizar la expresión, puede combinarse con AS para asignar un nombre a la columna resultante.

4. Consulta ejecutada:

SELECT *, CASE WHEN species = 'human' THEN 'talk' WHEN species = 'dog' THEN 'bark' ELSE 'meow' END AS sound FROM friends_of_pickles;

5. Resultado:



6. Interpretación: La consulta devuelve siete registros de la tabla friends_of_pickles y agrega una columna llamada sound, cuyo contenido depende de la especie de cada integrante, para humanos Dave y Mary se asigna el valor talk; para perros Odie, Jumpy y Sneakers se asigna bark; y para gatos Fry y Leela se devuelve meow, la expresión CASE permite clasificar registros mediante condiciones sin modificar la información original de la tabla.

7. Aprendizaje: Usar CASE para evaluar diferentes condiciones dentro de una consulta SQL y generar resultados personalizados según los valores de cada registro, comprendí cómo utilizar WHEN, THEN, ELSE y END para establecer reglas de clasificación, AS para asignar un nombre a la columna resultante.

8. Aplicación en el mundo real: Puede usarse en un sistema escolar para clasificar a los estudiantes como aprobados o reprobados según sus calificaciones, en una empresa para identificar empleados de acuerdo con su nivel salarial o en un sistema de ventas para clasificar productos según su disponibilidad en inventario.

Ejercicio 30. SUBSTR

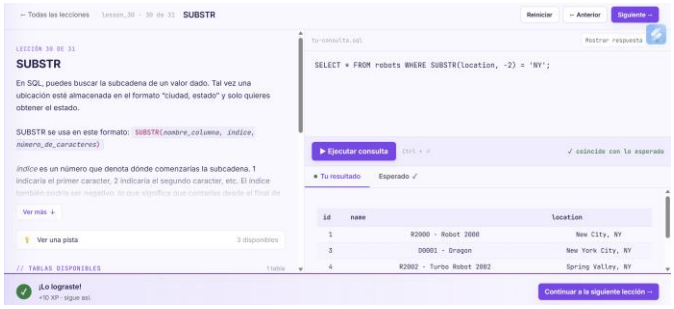
1. Número y nombre: Ejercicio 30 — SUBSTR.

2. Objetivo: Consultar registros de una tabla por la extracción de una parte específica de una cadena de texto, utilizando la función SUBSTR() para identificar los robots ubicados en el estado de Nueva York.

3. Concepto SQL: La función SUBSTR() permite extraer una parte de una cadena de texto almacenada en una columna, recibe como argumentos el nombre de la columna, la posición inicial y, opcionalmente, la cantidad de caracteres que se desean obtener, cuando se usa una posición negativa, la extracción comienza contando desde el final del texto, esta función puede combinarse con WHERE para filtrar registros según fragmentos específicos de información.

4. Consulta ejecutada: SELECT * FROM robots WHERE SUBSTR(location, -2) = 'NY';

5. Resultado:



6. Interpretación: La consulta devuelve cinco registros de la tabla robots cuyas ubicaciones terminan con las letras NY, los robots encontrados son Robot 2000, Dragon, Turbo Robot 2002, Super Robot 2003 y Unreleased Turbo Robot 2111, ubicados en diferentes ciudades del estado de Nueva York, la función SUBSTR() extrae los últimos dos caracteres de la columna location, mientras que la cláusula WHERE permite seleccionar únicamente los registros que coinciden con el valor NY.

7. Aprendizaje: Usar la función SUBSTR() para extraer fragmentos específicos de cadenas de texto dentro de una consulta SQL, comprendí cómo utilizar índices negativos para obtener caracteres desde el final de una cadena y combinarlos con condiciones de búsqueda para filtrar información de manera precisa.

8. Aplicación en el mundo real: Puede usarse en un sistema de logística para identificar envíos destinados a determinados estados, en una empresa para consultar clientes según códigos de ubicación o en un sistema de inventario para filtrar productos mediante fragmentos de sus códigos de identificación.

Ejercicio 31. COALESCE

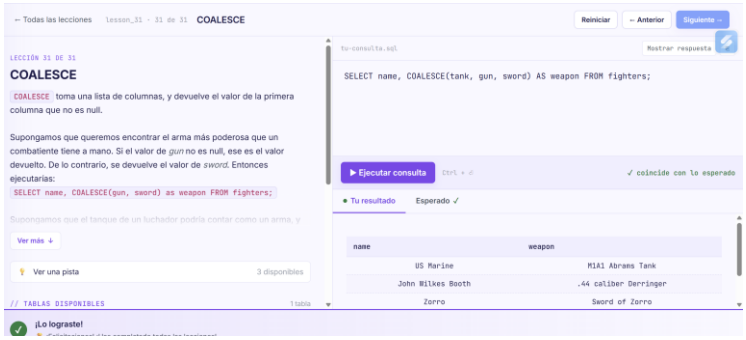
1. Número y nombre: Ejercicio 31 — COALESCE.

2. Objetivo: Consultar los registros de una tabla y seleccionar el primer valor no nulo entre varias columnas mediante la función COALESCE(), estableciendo un orden de prioridad para identificar el arma de cada luchador.

3. Concepto SQL: COALESCE() evalua una lista de valores o columnas y devuelve el primer valor que no sea NULL, los argumentos se evalúan de izquierda a derecha, por lo que su orden determina la prioridad, esa función resulta útil para manejar información incompleta y seleccionar valores alternativos cuando algunos campos no contienen datos.

4. Consulta ejecutada: SELECT name, COALESCE(tank, gun, sword) AS weapon FROM fighters;

5. Resultado:



6. Interpretación: Obtiene los nombres de luchadores registrados en la tabla fighters y determina el arma correspondiente a cada uno mediante la función COALESCE(); primero verifica si existe un valor en la columna tank; si este es NULL, consulta la columna gun y, si también es NULL, utiliza el valor de sword, el resultado muestra una columna llamada weapon, que contiene el primer valor disponible según el orden de prioridad establecido.

7. Aprendizaje: COALESCE() se usa para manejar valores NULL dentro de una consulta SQL, comprendí cómo establecer prioridades entre diferentes columnas y seleccionar automáticamente el primer valor disponible, evitando depender de una sola columna cuando existen datos incompletos.

8. Aplicación en el mundo real: Puede usarse en un sistema de gestión de clientes para seleccionar un medio de contacto prioritario entre teléfono, correo electrónico y dirección; en una empresa para consultar información alternativa cuando faltan determinados datos; o en un sistema de inventario para identificar proveedores disponibles según un orden de preferencia.

Reflexión y conclusión de los 31 ejercicios SQL

Durante el desarrollo de los 31 ejercicios de SQL Easy, reforcé mis conocimientos sobre el funcionamiento de las bases de datos relacionales y la importancia de utilizar consultas para obtener, organizar y analizar información, por en el periodos de enero-junio 2026 justo había llevado esa materia, por eos no se me hizo tan difícil, entonces volví a usar instrucciones como SELECT, WHERE, ORDER BY,

GROUP BY y diferentes funciones de agregación para realizar operaciones sobre los registros almacenados en una base de datos.

Los temas que me resultaron más sencillos fueron SELECT, WHERE, AND, OR y ORDER BY, debido a que sus funciones son directas y permiten observar fácilmente los cambios en los resultados; pero los ejercicios que más díficles fueron de consultas anidadas, los múltiples JOIN y los SELF JOIN, ya que requieren comprender las relaciones entre las tablas, identificar correctamente las columnas y establecer las condiciones necesarias para combinar la información, además esas operaciones igual me costaron cuando llevé la materia de Administración de Bases de Datos, fue una gran ayuda porque no estuve en 0 a la hora de ver esto en la sesiones o a la hora de realizar estos ejercicios.

Estos conocimientos pueden aplicarse en situaciones reales, como sistemas escolares, plataformas de comercio electrónico, aplicaciones empresariales y sistemas de inventario; mediante SQL es posible consultar estudiantes registrados, identificar productos disponibles, calcular promedios, generar estadísticas y relacionar información almacenada en diferentes tablas.

En conclusión, esta actividad me permitió desarrollar y mejorar habilidades prácticas para construir e interpretar consultas SQL, comprender mejor la estructura de las bases de datos y reconocer su importancia en el desarrollo de sistemas informáticos, volver a ver temas que sentía que no había comprendido del todo bien y además, considero que estos conocimientos serán fundamentales para diseñar aplicaciones que administren información de manera organizada y eficiente.

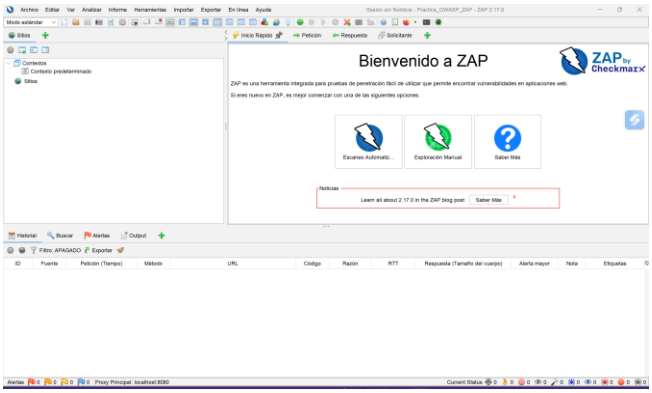
1. Preparación del entorno

1.1. Instalación e interfaz principal de OWASP ZAP

Objetivo: Instalar y ejecutar OWASP ZAP para preparar el entorno de análisis de seguridad de aplicaciones web.

Descripción: Se realizó la instalación de OWASP ZAP 2.17.0 en un equipo con Windows, después se inició el programa y se accedió a su interfaz principal en modo estándar, se identificaron las principales secciones de trabajo, como Sitios, Inicio Rápido, Historial y Alertas, que permitirán explorar una aplicación web, registrar el tráfico HTTP/HTTPS y analizar posibles vulnerabilidades.

Evidencia:



1.2. Configuración del proxy local

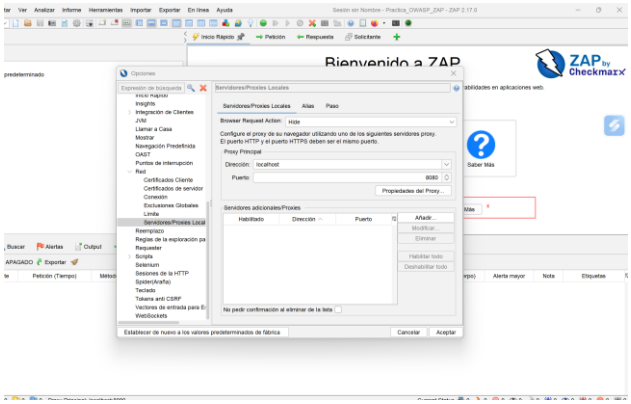
Objetivo: Verificar la configuración del proxy local de OWASP ZAP para permitir la captura y el análisis del tráfico HTTP/HTTPS generado por el navegador.

Descripción: Se accedió al menú Herramientas, seguido de Opciones y la configuración de red, para revisar los parámetros del proxy principal de OWASP ZAP, se verificaron la dirección local y el puerto utilizados por la herramienta, los cuales permiten establecer la comunicación entre el navegador y ZAP para registrar las solicitudes y respuestas de una aplicación web.

Configuración verificada:

- Dirección: localhost.
- Puerto: 8080.

Evidencia:



Posteriormente se utilizó la opción Exploración Manual de OWASP ZAP para iniciar Google Chrome con la configuración del proxy integrada automáticamente, así las solicitudes HTTP/HTTPS generadas durante la navegación fueron dirigidas al proxy local de ZAP, configurado en localhost y el puerto 8080. El funcionamiento de esta conexión se comprobó mediante la aparición de solicitudes en la pestaña Historial de la herramienta.

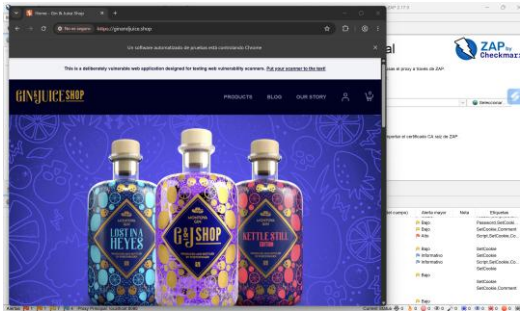
1.3. Acceso a Gin & Juice Shop

Objetivo: Acceder a la aplicación web Gin & Juice Shop mediante un navegador configurado desde OWASP ZAP para iniciar la exploración de seguridad.

Descripción: Se usó la opción Exploración Manual de OWASP ZAP para iniciar Google Chrome y acceder a la aplicación Gin & Juice Shop mediante la dirección https://ginandjuice.shop/.

Después de actualizar el navegador para solucionar un problema de compatibilidad con ChromeDriver, se logró establecer la conexión y visualizar correctamente la página principal de la aplicación, esta actividad permitió preparar el entorno para registrar el tráfico HTTP/HTTPS y realizar el análisis de seguridad.

Evidencia:



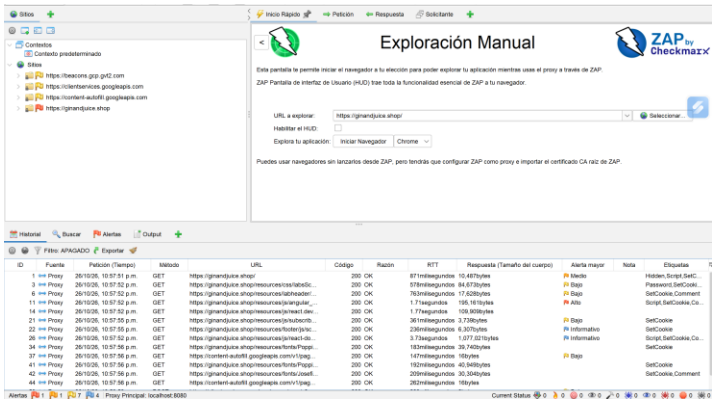
1.4. Verificación del tráfico HTTP/HTTPS

Objetivo: Comprobar que OWASP ZAP registra las solicitudes y respuestas HTTP/HTTPS generadas durante la navegación por Gin & Juice Shop.

Descripción: Se utilizó la pestaña Historial de OWASP ZAP para revisar las solicitudes generadas al acceder a Gin & Juice Shop, se observaron los métodos HTTP, las direcciones URL solicitadas y los

códigos de respuesta del servidor, esto permitió comprobar el funcionamiento del proxy y conocer la comunicación establecida entre el navegador y la aplicación web.

Evidencia:



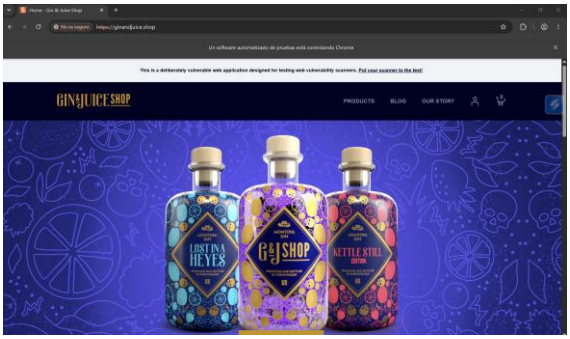
2. Exploración de la aplicación web

2.1. Navegación por la página principal

Objetivo: Explorar la interfaz principal de Gin & Juice Shop para identificar sus secciones y funcionalidades disponibles.

Descripción: Se realizó una exploración inicial de la página principal de Gin & Juice Shop usando Google Chrome configurado mediante OWASP ZAP. Se identificaron diferentes elementos de navegación, como la sección de productos, el blog, la información de la tienda, el acceso a usuarios y el carrito de compras, esta exploración permitió conocer la estructura de la aplicación e identificar las funcionalidades que serán revisadas durante el análisis de seguridad.

Evidencia:



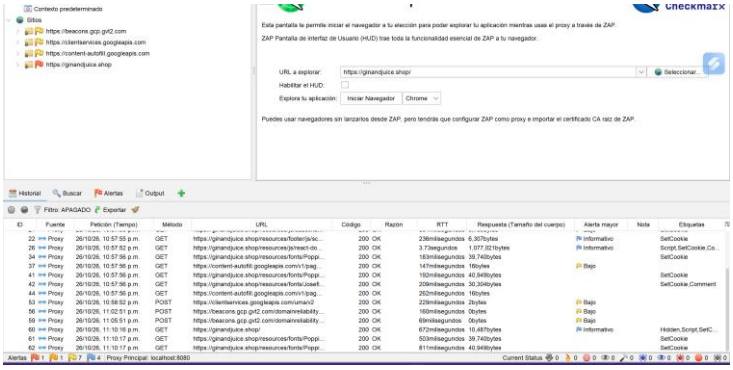
2.2. Tráfico HTTP generado durante la navegación

Objetivo: Analizar las solicitudes HTTP/HTTPS generadas durante la navegación por Gin & Juice Shop mediante el historial de OWASP ZAP.

Descripción: Se utilizó la pestaña Historial de OWASP ZAP para observar las solicitudes generadas durante la navegación por Gin & Juice Shop. Se identificaron peticiones mediante los métodos GET y POST, así como respuestas del servidor con el código HTTP 200 (OK), también se observaron indicadores

de alertas de seguridad y solicitudes a recursos como archivos JavaScript y fuentes, esta actividad permitió comprobar el funcionamiento del proxy y analizar la comunicación entre el navegador y la aplicación web.

Evidencia:

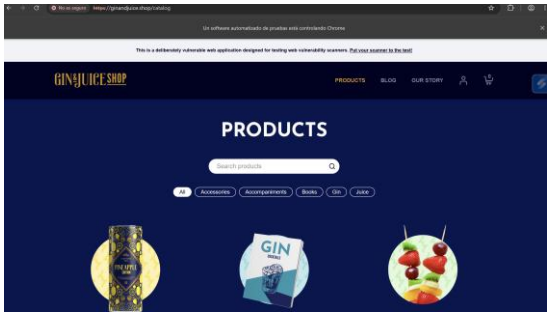


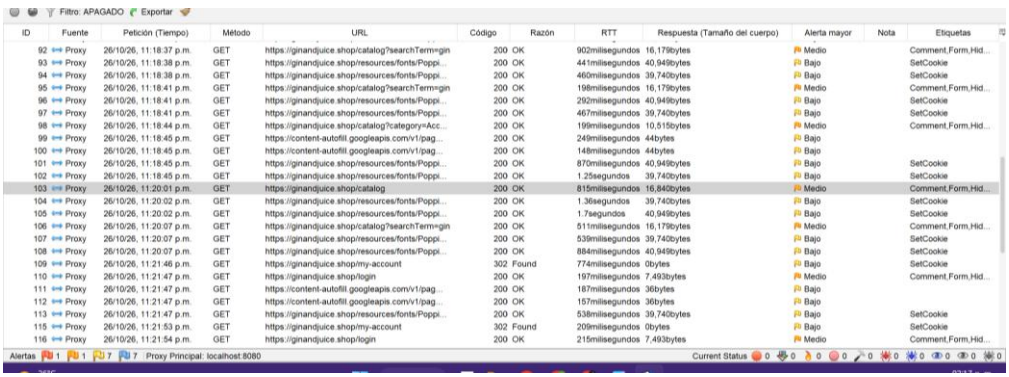
2.3. Exploración de la sección de productos

Objetivo: Explorar el catálogo de productos de Gin & Juice Shop para identificar su estructura y observar las funcionalidades disponibles.

Descripción: Se accedió a la sección PRODUCTS de Gin & Juice Shop mediante el navegador configurado desde OWASP ZAP, se exploró el catálogo de productos, identificando la información y los elementos de navegación disponibles, esta actividad permitió observar el funcionamiento de una sección adicional de la aplicación y generar nuevas solicitudes HTTP/HTTPS que pueden analizarse mediante OWASP ZAP.

Evidencia:



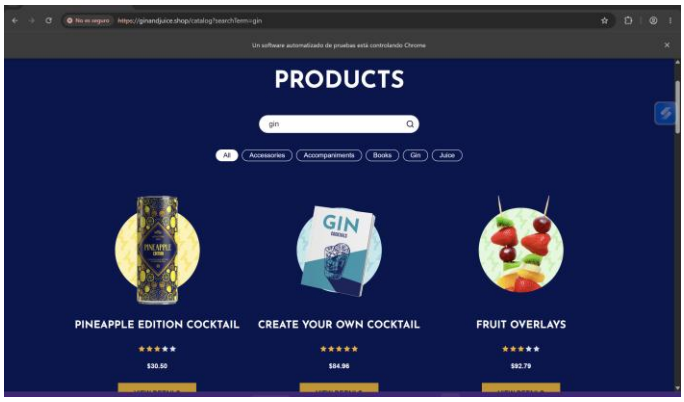


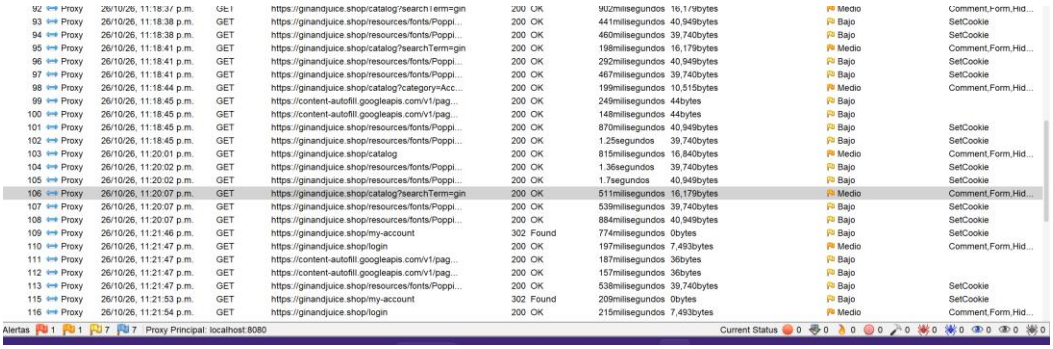
2.4. Exploración de la búsqueda de productos

Objetivo: Examinar el funcionamiento del buscador de productos de Gin & Juice Shop mediante la introducción de un término de búsqueda, para observar los resultados y la información enviada por la aplicación.

Descripción: Se accedió al catálogo de Gin & Juice Shop y se utilizó el campo Search products para introducir la palabra gin, se observó que la dirección URL incorporó el parámetro searchTerm=gin, correspondiente al término de búsqueda ingresado, esta actividad permitió conocer el funcionamiento de la interfaz de búsqueda y generar tráfico HTTP/HTTPS que puede revisarse mediante OWASP ZAP.

Evidencia:



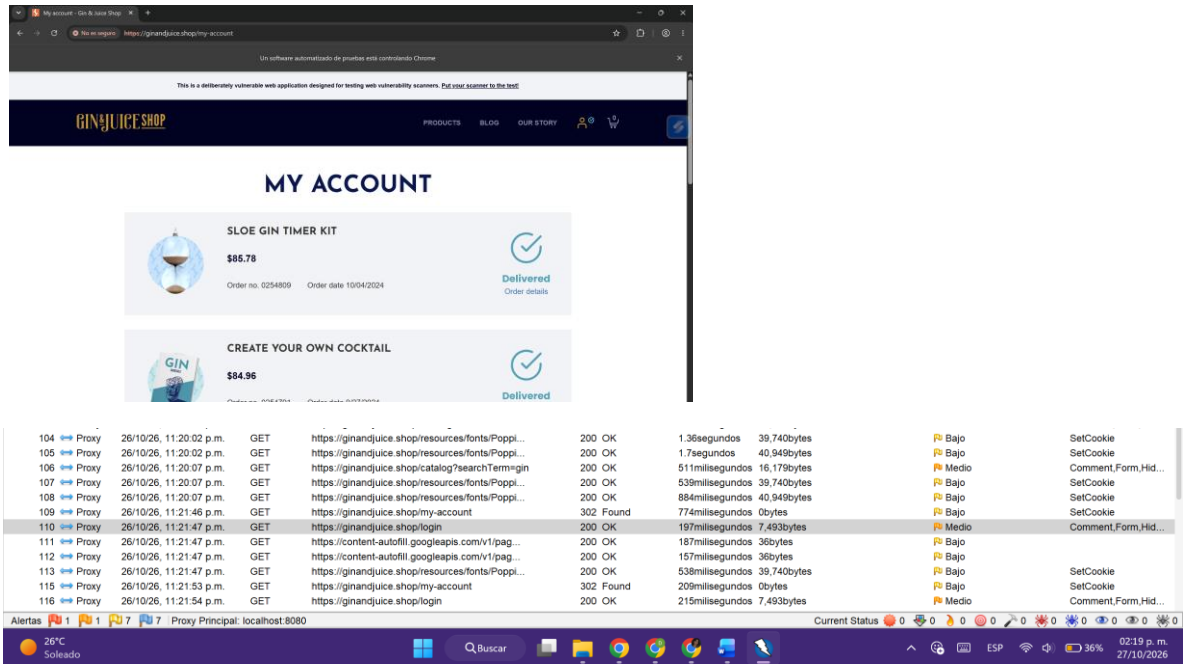


2.5. Exploración del inicio de sesión y cuenta de usuario

Objetivo: Explorar el proceso de autenticación de Gin & Juice Shop y verificar el acceso a una cuenta de usuario mediante credenciales de prueba, utilizando un navegador configurado desde OWASP ZAP.

Descripción: Se accedió a la sección de inicio de sesión de Gin & Juice Shop y se utilizaron credenciales de demostración para ingresar a la aplicación, después de la autenticación, se obtuvo acceso a la página MY ACCOUNT, donde se visualizó información relacionada con los pedidos del usuario, incluyendo productos, precios, fechas y estados de entrega, esa actividad permitió comprobar el acceso a una sección de usuario y generar tráfico HTTP/HTTPS relacionado con la autenticación y la consulta de información de la cuenta.

Evidencia:



3. Análisis de alertas de seguridad

Objetivo: Identificar y analizar tres tipos diferentes de alertas detectadas por OWASP ZAP durante la exploración de Gin & Juice Shop, considerando su nivel de riesgo, descripción y posibles consecuencias para la seguridad de la aplicación.

Descripción: Después de explorar las diferentes secciones de Gin & Juice Shop, se accedió a la pestaña Alertas de OWASP ZAP, la herramienta mostró 16 tipos de alertas relacionados con bibliotecas JavaScript, encabezados de seguridad, cookies y otros aspectos de la aplicación web, para realizar el análisis se seleccionaron tres tipos diferentes de alertas, con el propósito de comprender sus características, sus posibles riesgos y las recomendaciones para su mitigación.

3.1. Alerta 1 — Librería JS Vulnerable

Objetivo: Identificar y analizar una alerta de seguridad relacionada con el uso de una biblioteca JavaScript vulnerable en la aplicación Gin & Juice Shop, utilizando los resultados proporcionados por OWASP ZAP.

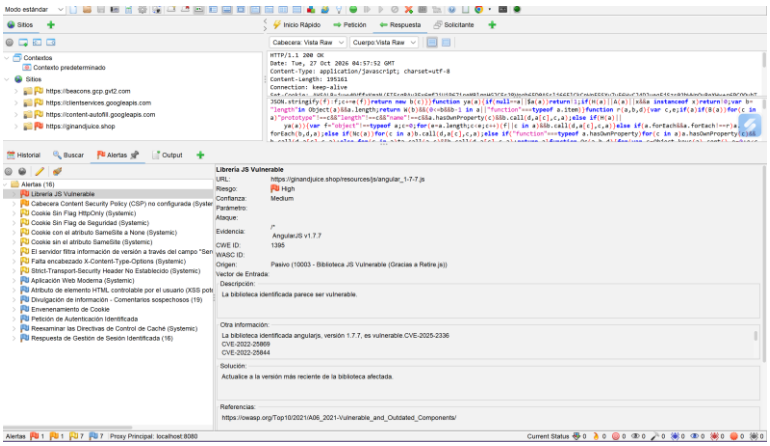
Descripción: Durante la exploración de Gin & Juice Shop, OWASP ZAP identificó una alerta denominada «Librería JS Vulnerable», clasificada con un nivel de riesgo alto y confianza media, la herramienta detectó la utilización de AngularJS en su versión 1.7.7 dentro de un archivo JavaScript de la aplicación, esta versión pertenece a una biblioteca que ha finalizado su soporte oficial y está asociada con vulnerabilidades conocidas.

Datos de la alerta:

- Nombre: Librería JS Vulnerable.
- Nivel de riesgo: Alto.

- Nivel de confianza: Medio.
- Biblioteca afectada: AngularJS.
- Versión detectada: 1.7.7.
- URL: https://ginandjuice.shop/resources/js/angular_1-7-7.js.
- Tipo de análisis: Pasivo.
- CWE ID: 1395.

Evidencia:



Interpretación: La alerta indica que la aplicación utiliza una versión de AngularJS asociada con problemas de seguridad conocidos, OWASP ZAP identificó la versión de la biblioteca a partir del contenido del archivo JavaScript y la comparó con información sobre componentes vulnerables, este resultado representa un posible riesgo, aunque no demuestra por sí solo que las vulnerabilidades puedan explotarse en la aplicación analizada.

Impacto potencial: El uso de bibliotecas JavaScript vulnerables puede exponer a una aplicación a problemas como ejecución de scripts no deseados, manipulación de contenido o afectaciones al rendimiento, dependiendo de las vulnerabilidades presentes y de cómo se utilice el componente.

Recomendación: Se recomienda revisar las dependencias JavaScript utilizadas por la aplicación, verificar las vulnerabilidades asociadas con sus versiones y sustituir AngularJS por una alternativa que cuente con soporte y actualizaciones de seguridad, también se deben aplicar controles para validar las entradas y reducir la exposición a posibles ataques.

3.2. Alerta 2 — Cabecera Content Security Policy (CSP) no configurada

Objetivo: Identificar y analizar una alerta de seguridad relacionada con la ausencia del encabezado Content Security Policy (CSP) en Gin & Juice Shop, utilizando los resultados obtenidos mediante OWASP ZAP.

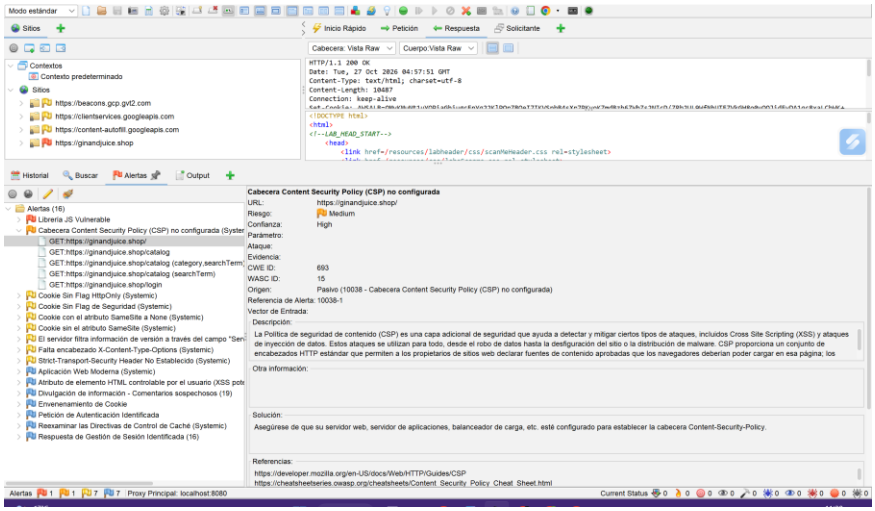
Descripción: Durante el análisis de Gin & Juice Shop, OWASP ZAP identificó una alerta denominada «Cabecera Content Security Policy (CSP) no configurada», clasificada con un nivel de riesgo medio y confianza alta, la herramienta detectó que la respuesta HTTP de la página principal no contiene el

encabezado Content-Security-Policy, que permite establecer restricciones sobre los recursos que puede cargar y ejecutar un navegador, su ausencia reduce las medidas de protección de la aplicación frente a determinados ataques web.

Datos de la alerta:

- Nombre: Cabecera Content Security Policy (CSP) no configurada.
- Nivel de riesgo: Medio.
- Nivel de confianza: Alto.
- URL afectada: https://ginandjuice.shop/
- Tipo de análisis: Pasivo.
- CWE ID: 693.
- WASC ID: 15.

Evidencia:



Interpretación: La alerta indica que la aplicación Gin & Juice Shop no establece una política de seguridad de contenido mediante el encabezado HTTP Content-Security-Policy en la respuesta analizada, esta configuración permite a los desarrolladores controlar los orígenes desde los cuales una página puede cargar recursos, como scripts, estilos e imágenes, la ausencia del encabezado disminuye la protección adicional frente a ciertos ataques, aunque no demuestra por sí misma la existencia de una vulnerabilidad explotable.

Impacto potencial: La falta de una política CSP puede facilitar que determinados ataques de inyección de contenido tengan mayores consecuencias si existen otras vulnerabilidades en la aplicación, entre los riesgos potenciales se encuentran la ejecución de scripts no autorizados, la manipulación del contenido de la página y la exposición de información.

Recomendación: Se recomienda configurar el encabezado HTTP Content-Security-Policy en el servidor web, estableciendo reglas que permitan cargar recursos únicamente desde fuentes autorizadas,

también es importante revisar las dependencias externas, restringir la ejecución de scripts no confiables y realizar pruebas para comprobar que la política implementada protege la aplicación sin afectar su funcionamiento.

3.3. Alerta 3 — Cookie Sin Flag HttpOnly

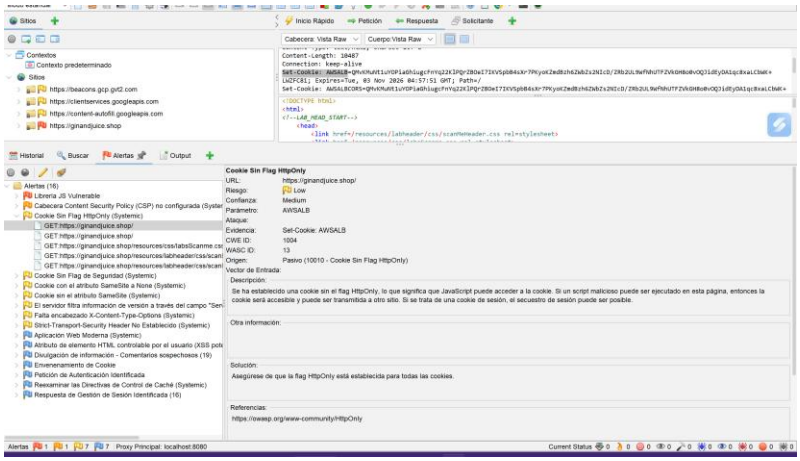
Objetivo: Identificar y analizar una alerta de seguridad relacionada con la ausencia del atributo HttpOnly en las cookies utilizadas por Gin & Juice Shop, mediante los resultados obtenidos con OWASP ZAP.

Descripción: Durante el análisis de Gin & Juice Shop, OWASP ZAP identificó una alerta denominada «Cookie Sin Flag HttpOnly», clasificada con un nivel de riesgo bajo y confianza media, la herramienta detectó que algunas cookies enviadas por el servidor, como AWSALB y AWSALBCORS, no presentan el atributo HttpOnly, este atributo permite restringir el acceso a las cookies mediante JavaScript, proporcionando una medida adicional de protección para la información almacenada en ellas.

Datos de la alerta:

- Nombre: Cookie Sin Flag HttpOnly.
- Nivel de riesgo: Bajo.
- Nivel de confianza: Medio.
- URL afectada: https://ginandjuice.shop/
- Parámetro afectado: AWSALB.
- Evidencia técnica: Set-Cookie: AWSALB.
- Tipo de análisis: Pasivo.
- CWE ID: 1004.
- WASC ID: 13.

Evidencia:



Interpretación: La alerta indica que la cookie AWSALB no presenta el atributo HttpOnly en la respuesta HTTP analizada, significa que su contenido podría ser accesible mediante JavaScript ejecutado en el navegador, aunque la ausencia de este atributo no demuestra por sí sola una vulnerabilidad explotable, representa una configuración que debe revisarse para determinar si la cookie contiene información sensible y necesita protección adicional.

Impacto potencial: Si una cookie utilizada para autenticación o gestión de sesiones carece del atributo HttpOnly, un atacante que consiga ejecutar JavaScript malicioso en la aplicación podría acceder a su contenido, dependiendo de la función de la cookie y de los controles de seguridad existentes, esto podría facilitar el acceso no autorizado a información de sesión.

Recomendación: Se recomienda revisar la configuración de las cookies utilizadas por la aplicación y establecer el atributo HttpOnly en aquellas que no requieren acceso mediante JavaScript, especialmente las relacionadas con autenticación y sesiones, también es importante utilizar atributos como Secure y SameSite cuando corresponda, para fortalecer la protección de las cookies y reducir posibles riesgos de seguridad.

4. Documentación del hallazgo principal

Hallazgo seleccionado: Librería JS Vulnerable — AngularJS 1.7.7

1. Nombre de la alerta: Librería JS Vulnerable (AngularJS 1.7.7).

2. Nivel de riesgo: Alto (High), con un nivel de confianza medio (Medium), según la clasificación proporcionada por OWASP ZAP.

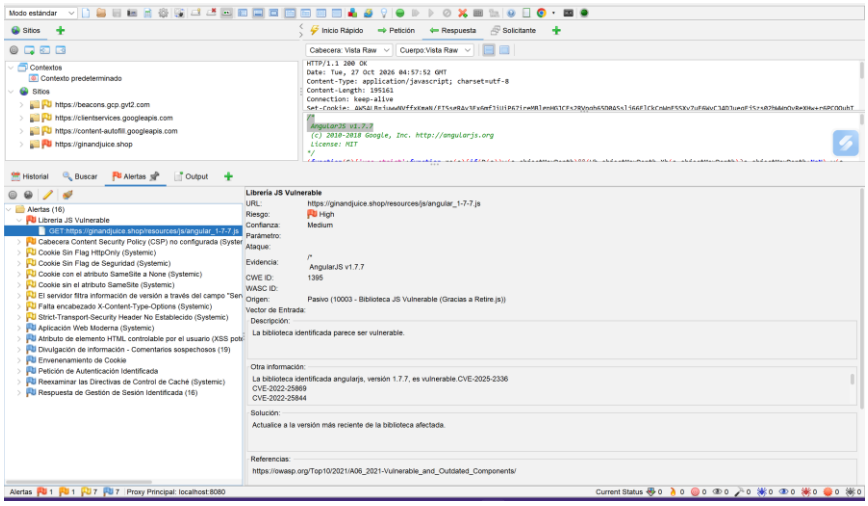
3. URL o componente afectado: La alerta se identificó en el archivo JavaScript de la aplicación Gin & Juice Shop, ubicado en la siguiente dirección:

https://ginandjuice.shop/resources/js/angular_1-7-7.js

El componente identificado corresponde a la biblioteca AngularJS, versión 1.7.7.

4. Parámetro afectado: No aplica, debido a que la alerta no está relacionada con un parámetro específico de entrada, sino con la utilización de una biblioteca JavaScript asociada con vulnerabilidades conocidas.

5. Evidencia:



6. Descripción técnica: El hallazgo indica que Gin & Juice Shop utiliza una versión de AngularJS que presenta vulnerabilidades conocidas, OWASP ZAP identificó la versión de la biblioteca mediante el análisis del archivo JavaScript y la relacionó con información de seguridad registrada en su base de detección, la utilización de componentes vulnerables o sin mantenimiento puede incrementar la exposición de una aplicación a ataques informáticos, pero esta detección no confirma por sí sola que las vulnerabilidades puedan explotarse en el sistema analizado.

7. Impacto potencial: El uso de una biblioteca JavaScript vulnerable podría facilitar ataques como Cross-Site Scripting (XSS), manipulación de contenido o denegación de servicio, dependiendo de las vulnerabilidades aplicables y de cómo se utilice la biblioteca en la aplicación, estos problemas podrían afectar la confidencialidad, integridad o disponibilidad del sistema, especialmente cuando existen otras debilidades de seguridad.

8. Recomendación: Se recomienda revisar las dependencias JavaScript utilizadas por Gin & Juice Shop e identificar los componentes que presentan vulnerabilidades conocidas, aunque se sugiere sustituir AngularJS 1.7.7 por una tecnología que cuente con mantenimiento y actualizaciones de seguridad, debido a que AngularJS ha finalizado su soporte oficial, también se recomienda implementar controles de validación de entradas, fortalecer las políticas de seguridad del navegador y realizar evaluaciones periódicas para detectar componentes vulnerables.

5. CONCLUSIONES

5.1. ¿Qué aprendí?

Durante esta actividad aprendí a usar OWASP ZAP como herramienta para analizar la seguridad de aplicaciones web, comprendí mejor el cómo funciona un proxy y cómo permite observar las solicitudes y respuestas HTTP/HTTPS generadas durante la navegación, también aprendí a identificar e interpretar alertas de seguridad, reconociendo problemas como el uso de bibliotecas JavaScript vulnerables, la ausencia de políticas de seguridad de contenido (CSP) y la falta del atributo HttpOnly en las cookies, en esta práctica me permitió comprender la importancia de identificar posibles vulnerabilidades y aplicar medidas preventivas para proteger las aplicaciones web.

5.2. ¿Qué fue lo más retador?

Uno de los principales retos fue configurar correctamente el entorno de trabajo, ya que inicialmente OWASP ZAP no podía iniciar Google Chrome debido a un problema de compatibilidad con el controlador del navegador, tuve que revisar los complementos Selenium y Windows WebDrivers y actualicé Google Chrome, porque no estaba actualizado, o bueno era algo como reincio, y así logré establecer la conexión con ZAP, otro desafío fue comprender la información técnica de las alertas, ya había visto algo parecido en la escuela, pero solo en una clase, aquí pude verlo y entenderlo mejor, como el identificar sus niveles de riesgo e interpretar las posibles consecuencias de cada hallazgo, estas dificultades me ayudaron a desarrollar habilidades de resolución de problemas y análisis de seguridad.

5.3. ¿Qué revisaría a continuación?

Como continuación de esta práctica, profundizaría en el análisis de las vulnerabilidades identificadas para comprobar su alcance y determinar las medidas necesarias para corregirla, revisaría especialmente las bibliotecas JavaScript desactualizadas, la configuración de los encabezados de seguridad y la protección de las cookies, también realizaría nuevas pruebas autorizadas sobre los mecanismos de autenticación, la validación de entradas y la gestión de sesiones, con el propósito de comprender mejor los riesgos y fortalecer la seguridad de las aplicaciones web.